

Week-5

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Ans:- Traditional MapReduce processes data in multiple stages, and after each stage it writes intermediate results to disk. This increases the execution time because reading from and writing to disk is much slower than processing data in memory. MapReduce is also not well suited for iterative tasks such as machine learning algorithms and interactive data analysis, where the same data needs to be processed repeatedly.

Apache Spark overcomes these limitations by processing data in memory whenever possible. This significantly reduces execution time and makes Spark much faster than MapReduce. Spark also provides built-in libraries for SQL, machine learning, graph processing, and streaming, making it a better choice for modern big data applications.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Ans:- Spark stores intermediate data in memory (RAM) instead of writing it to disk after every operation. In machine learning algorithms, the same dataset is processed many times to improve the model. Since the data is already available in memory, Spark does not need to read it from disk repeatedly, which saves a lot of time.

In contrast, disk-based systems like MapReduce read and write data to disk after every iteration, making the execution much slower. Therefore, Spark's in-memory computing improves the performance of iterative algorithms and enables faster data processing

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

```

Rows before removing duplicates: 20
Rows after removing duplicates: 19
26/07/19 22:42:03 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|transaction_date|region|product_category|sale_amount|status|city|age|subscription|raw_timestamp|email|username|price|st
ore_id|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|101|2025-07-01|West|Electronics|55000|Completed|Jaipur|25|Premium|2025-07-01 10:15:00|amit.sharma@gmail.com|amit01|55000|S1
|102|2025-07-02|North|Furniture|12000|Pending|Delhi|31|Basic|2025-07-02 11:00:00|priya.verma@gmail.com|priya02|12000|S1
|103|2025-07-03|South|Groceries|2500|Completed|Bengaluru|22|Premium|2025-07-03 09:20:00|rahul.kumar@gmail.com|NULL|2500|S1
|104|2025-07-04|East|Clothing|3500|NULL|Kolkata|28|Basic|2025-07-04 14:00:00|neha.singh@gmail.com|neha04|3500|S1
|105|2025-07-05|West|Electronics|72000|Completed|Mumbai|35|Premium|2025-07-05 16:30:00|vivek.patel@gmail.com|vivek05|72000|S1
|106|2025-07-06|North|Furniture|18000|Cancelled|Chandigarh|40|Basic|2025-07-06 12:00:00|NULL|rohit06|18000|S1

```

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

```

+-----+-----+
|product_category|average_sale_amount|
+-----+-----+
|Electronics|65000.0|
+-----+-----+

```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

```

+-----+-----+
|user_id|status|
+-----+-----+
|101|Completed|
|101|Completed|
|102|Pending|
|103|Completed|
|104|Unknown|
|105|Completed|
|106|Cancelled|
|107|Completed|
|108|Pending|
|109|Completed|
|110|Completed|
|111|Pending|
|112|Completed|
|113|Completed|
|114|Pending|
|115|Unknown|
|116|Completed|
|117|Cancelled|
|118|Pending|
|119|Completed|
+-----+-----+

```

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
+----+-----+
|city|count|
+----+-----+
+----+-----+
```

The sample dataset contains only 20 records, so no city has more than 100 records. Therefore, the query returns an empty result

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Ans :- Spark DataFrames are immutable, which means they cannot be modified after they are created. Whenever we perform operations such as dropping a column, renaming a column, or removing duplicates, Spark creates a new DataFrame instead of changing the original one.

For example, if we remove duplicate records, the original DataFrame remains unchanged and a new cleaned DataFrame is returned. This approach makes data processing more reliable because the original data is always preserved.

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

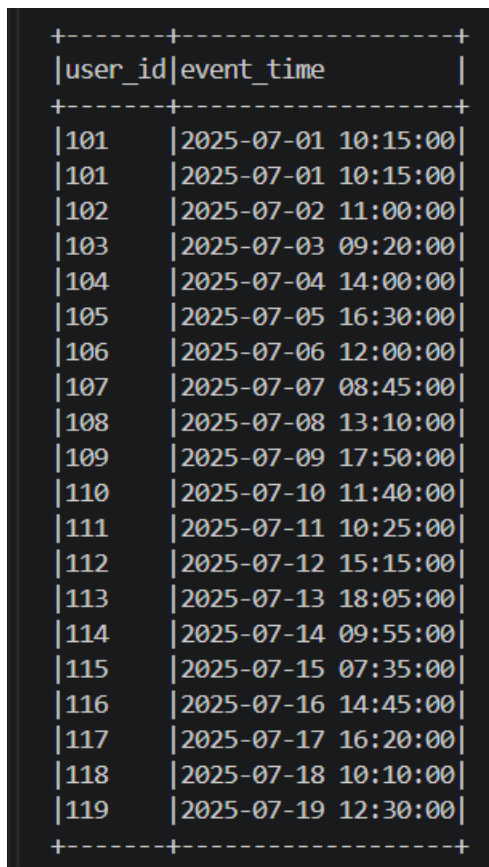
```
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|transaction_date|region|product_category|sale_amount|status|city|age|subscription|raw_timestamp|email|username|price|store_id|
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|101|2025-07-01|West|Electronics|55000|Completed|Jaipur|25|Premium|2025-07-01 10:15:00|amit.sharma@gmail.com|amit01|55000|S101|
|101|2025-07-01|West|Electronics|55000|Completed|Jaipur|25|Premium|2025-07-01 10:15:00|amit.sharma@gmail.com|amit01|55000|S101|
|103|2025-07-03|South|Groceries|2500|Completed|Bengaluru|22|Premium|2025-07-03 09:20:00|rahul.kumar@gmail.com|NULL|2500|S103|
|107|2025-07-07|South|Groceries|1800|Completed|Hyderabad|19|Premium|2025-07-07 08:45:00|kavya.reddy@gmail.com|kavya07|1800|S103|
|109|2025-07-09|West|Electronics|68000|Completed|Pune|29|Premium|2025-07-09 17:50:00|sneha.joshi@gmail.com|sneha09|68000|S101|
|111|2025-07-11|South|Groceries|3000|Pending|Chennai|24|Premium|2025-07-11 10:25:00|pooja.nair@gmail.com|pooja11|3000|S103|
|115|2025-07-15|South|Groceries|2200|NULL|Kochi|21|Premium|2025-07-15 07:35:00|ananya.pillai@gmail.com|ananya15|2200|S103|
|119|2025-07-19|South|Groceries|2800|Completed|Mysuru|20|Premium|2025-07-19 12:30:00|shruti.hegde@gmail.com|shruti19|2800|S103|
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Ans :- Handling null values before performing calculations helps produce more accurate and meaningful results. If null values are ignored or left untreated, they may affect the output of operations such as `sum()`, `avg()`, or other statistical calculations.

By filling missing values with suitable defaults or removing incomplete records, we ensure that the calculations are based on valid data. This improves the quality and reliability of the analysis.

Q10: Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.



user_id	event_time
101	2025-07-01 10:15:00
101	2025-07-01 10:15:00
102	2025-07-02 11:00:00
103	2025-07-03 09:20:00
104	2025-07-04 14:00:00
105	2025-07-05 16:30:00
106	2025-07-06 12:00:00
107	2025-07-07 08:45:00
108	2025-07-08 13:10:00
109	2025-07-09 17:50:00
110	2025-07-10 11:40:00
111	2025-07-11 10:25:00
112	2025-07-12 15:15:00
113	2025-07-13 18:05:00
114	2025-07-14 09:55:00
115	2025-07-15 07:35:00
116	2025-07-16 14:45:00
117	2025-07-17 16:20:00
118	2025-07-18 10:10:00
119	2025-07-19 12:30:00

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Ans :- A shuffle is the process of redistributing data across different partitions so that records with the same key are brought together. During operations like `groupBy()`, Spark moves data between different executors to group similar values.

Shuffle is considered a wide transformation because data is exchanged across multiple partitions instead of being processed within a single partition. Since moving data over the network is expensive, shuffle operations usually take more time than narrow transformations

Q12: Write a code snippet that identifies and **removes rows** where the `email` column contains `null` values OR the `username` is an empty string.

```
Rows before cleaning: 20
Rows after cleaning: 18
```

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the `min`, `max`, and `mean` of the `price` column?

```
+-----+-----+-----+
|Minimum Price|Maximum Price|Average Price|
+-----+-----+-----+
|          1800|          81000|        26700.0|
+-----+-----+-----+
```

Spark calculated the minimum, maximum, and average price in a single aggregation. Null values in the `price` column were automatically ignored while calculating the average.

Q14: In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

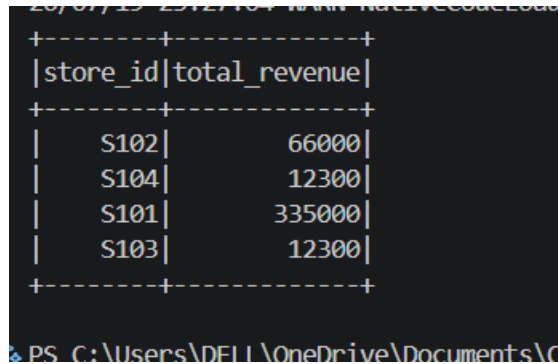
Ans:- When `inferSchema=true` is used, Spark automatically determines the data type of each column by examining the input data. If a date column contains inconsistent formats, Spark may infer the wrong data type or treat the entire column as a string. In some cases, it may also produce null values for invalid dates.

To avoid these issues, it is better to define the schema explicitly or clean and standardize the date formats before loading the data.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.

3. Groups by store_id to calculate total revenue.



A terminal window with a dark background. At the top, there is a header line: 28/07/19 23:12:18 C:\Windows\system32\cmd.exe. Below this, a table is displayed with two columns: store_id and total_revenue. The table has four data rows. The first row shows S102 with a total revenue of 66000. The second row shows S104 with a total revenue of 12300. The third row shows S101 with a total revenue of 335000. The fourth row shows S103 with a total revenue of 12300. The table is enclosed in a box with dashed lines and plus signs at the corners. At the bottom of the terminal window, the command prompt path is visible: PS C:\Users\DELL\OneDrive\Documents\G

store_id	total_revenue
S102	66000
S104	12300
S101	335000
S103	12300

The pipeline first removes duplicate records to avoid counting the same transaction multiple times. It then replaces missing prices with 0 to ensure accurate calculations. Finally, the data is grouped by store_id to calculate the total revenue for each store.